Homework 1 — Part A: Basic Questions

Below are four faulty programs. Each includes test inputs that result in failure.
Answer the following questions about each program.

```
/**
 * Find last index of element
 *
 * @param {Object[]} x - The array to search.
 * @param {number} y - The value to look for.
 *
 * @returns {number} Last index of y in x; -1 if absent.
 * @throws TypeError if x is not an array or y is not a
 *         number.
 */
function findLast(x, y) {
    if (!Array.isArray(x)) {
        throw new TypeError('The first parameter must be
an array');
    }
    if (typeof y !== 'number') {
        throw new TypeError('The second parameter must be
a number');
    }
    for (let i = x.length - 1; i > 0; i--) {
        if (x[i] === y) {
            return i;
        }
    }
    return -1;
}
// test: x = [2, 3, 5]; y = 2; Expected = 0
```

```
/**
 * Find last index of zero
 *
 * @param {Object[]} x - The array to search.
 *
 * @returns {number} Last index of 0 in x; -1 if absent.
 * @throws TypeError if x is not an array.
 */
function lastZero(x) {
    if (!Array.isArray(x)) {
        throw new TypeError('Not an array');
    }
    for (let i = 0; i < x.length; i++) {
        if (x[i] === 0) {
            return i;
        }
    }
    return -1;
}
// test: x = [0, 1, 0]; Expected = 2
```

```
/**
 * Count positive elements
 *
 * @param {Object[]} x - The array to search.
 *
 * @returns {number} Count of positive elements in x.
 * @throws TypeError if x is not an array.
 */
function countPositive(x) {
    if (!Array.isArray(x)) {
        throw new TypeError('Not an array');
    }
    let count = 0;
    for (let i = 0; i < x.length; i++) {
        if (x[i] >= 0) {
            count++;
        }
    }
    return count;
}
// test: x = [-4, 2, 0, 2]; Expcted = 2
```

```
/**
 * Count odd or postive elements
 *
 * @param {Object[]} x - The array to search.
 *
 * @return {number} Count of odd/positive values in x.
 * @throws TypeError if x is not an array.
 */
function oddOrPos(x) {
    if (!Array.isArray(x)) {
        throw new TypeError('Not an array');
    }
    let count = 0;
    for (let i = 0; i < x.length; i++) {
        if (x[i] % 2 === 1 || x[i] > 0) {
            count++;
        }
    }
    return count;
}
// test: x = [-3, -2, 0, 1, 4]; Expected = 3
```

(a) Explain what is wrong with the given code.
    Describe the fault precisely by proposing a modification to the code.
(b) If possible, give a test case that does not execute the fault.
    If not, briefly explain why not. (You need to give the same number of arguments.)
(c) If possible, give a test case that executes the fault, but does not result in an error state.
    If not, briefly explain why not. (You also need to answer expected and actual output.)
(d) If possible, give a test case that results in an error state, but not a failure.
    If not, briefly explain why not. (You also need to answer expected and actual output.)
(e) For the given test case in (d), describe the first error state. Be sure to describe the complete state.

Hints:

Fault:	Error State:	Failure:
* A static defect in software. * Parts of source code that are incorrect.	* An incorrect internal state. * State information: variable values (including return value).	* External, incorrect behavior with respect to the expected behavior.

Homework 1 — Part B: Testing with an AI Coding Agent

Part B reuses the four programs from Part A. You will let an AI coding agent (for example Claude Code, GitHub Copilot, Cursor, Codex CLI, or Gemini CLI) write tests for them, and then judge those tests with the Fault / Error / Failure vocabulary and the **RIPR** model. **Finish Part A on your own, without an agent, before you start Part B.** Question B3 compares the agent's answers with yours.

Setup

1. Put the four functions in `hw1.js` and end the file with
   ```
   module.exports = { findLast, lastZero, countPositive, oddOrPos };
   ```
2. Put test files in `test/` with names ending in `.test.js`. Use Node's built-in test runner (Node 20 or later):
   ```
   node --test                                  # run all tests
   node --test --experimental-test-coverage     # tests + line / branch coverage
   ```
3. Start a **new agent session for every prompt**, so earlier code or answers cannot leak into it.

Hints:

Reachability: The location(s) in the program that contain the fault must be reached.	Infection: The state of the program must be incorrect.	Propagation: The infected state must cause some output or final state of the program to be incorrect.	Reveal: The tester must observe part of the incorrect portion of the final program state.
Assertion echo: The agent runs the current code and writes its output as the expected result. If the code is faulty, the fault becomes part of the test suite. An oracle needs a source of expected behavior that is independent of the implementation: a specification, a reference implementation, a property, or a metamorphic relation.			

B1. Assertion echo experiment

Run two prompts for each of the four programs.

- **Prompt A (code only).** Give the agent the whole function, including its body, and ask: *"Write unit tests for this function with node:test. Make sure all tests pass."*
- **Prompt B (spec only).** Give the agent only the JSDoc comment and the function header, with no body, and ask: *"Write unit tests with node:test that check this specification."*

Run every suite against the faulty code in `hw1.js` without changing it, and fill in the table.

Program	Prompt	# tests	# failing	Line %	Branch %	Fault revealed? (Y/N)
`findLast`	A					
	B					
`lastZero`	A					
	B					
`countPositive`	A					
	B					
`oddOrPos`	A					
	B					

(a) Find at least one assertion in the Prompt A suites whose expected value was copied from the faulty program's actual output. Quote it, give the correct expected value according to the JSDoc comment, and explain why this test passes today but will fail once the fault is fixed.

(b) Which prompt reached higher coverage, and which revealed more faults? In at most 150 words, explain why higher coverage does not guarantee that more faults are revealed.

B2. RIPR analysis of agent-generated tests

(a) For each program, pick one agent-generated test that does **not** fail on the faulty code. Name the **first** RIPR condition that the test does not satisfy, and justify it with the program state (variable values and return value).

Program	Test (quote the call and the assertion)	First condition not met (R / I / P / Reveal)	Justification
`findLast`			
`lastZero`			
`countPositive`			
`oddOrPos`			

At least one row must stop at **Reveal**: the fault is reached, the state is infected, and the wrong value reaches the return value, but the assertion does not look at it (for example `assert.ok(Number.isInteger(r))`). If the agent produced no such test, write one yourself and mark it "hand-written".

(b) The echoed assertion from B1(a) *does* look at the wrong return value, yet the test passes. Does it satisfy Reveal? Argue your answer in at most 100 words.

B3. Checking the agent's reasoning

Give a new agent session the code of `lastZero` together with Part A's questions (a)–(e), and ask it to answer them.

(a) Mark each of its answers **Correct**, **Incorrect**, or **Incomplete**. Run every test case it proposes and report the expected and actual output yourself; do not trust the outputs it claims.

(b) For (e), compare its first error state with yours. Did it describe the **complete** state: `x`, `i`, the return value, and the program counter (the statement that executes next)? List what is missing or wrong.

(c) If any answer is wrong, write one follow-up prompt that leads the agent to fix it **without giving away the answer**. Quote the prompt and report whether it worked.

B4. An oracle without a hard-coded expected value

Write one **metamorphic** or **property-based** test for `oddOrPos` that reveals the fault without writing any concrete expected count. *Hint: how should the count change when you append one more element to* `x`*?* Report the relation in words, the test code, and the failing output. You may ask an agent to write the code, but you must state the relation yourself.

B5. Reflection (at most 200 words)

(i) Which RIPR condition was hardest for the agent to satisfy, and why?

(ii) An agent-generated test is often a *snapshot of current behavior* rather than a *specification of expected behavior*. Name two independent sources of expected behavior you could give an agent to fix this, and say which one you used in B4.

Bonus (optional): mutation testing

Run StrykerJS on the Prompt A and Prompt B suites of one program (`npx stryker init`, choose the *command* test runner with the command `node --test`, then `npx stryker run`). Report both mutation scores, quote one surviving mutant, and name the RIPR condition that the suite failed to satisfy for that mutant.

Submission

- **Report (PDF):** answers to Part A and Part B, including the B1 and B2 tables.
- **Code (zip or your GitHub Classroom repository):** `hw1.js`; the agent's suites saved unchanged as `test/promptA_<program>.test.js` and `test/promptB_<program>.test.js`; your own tests as `test/mine_*.test.js`; and the coverage output.
- **AI usage log (required):** tool name and version, model, and date; every prompt verbatim; and the agent's full responses as an appendix. If you edit an agent's test, keep the original and mark your edit.
- **Grading** rewards oracle sources, RIPR reasoning, and careful verification of the agent. It does not reward the number of tests or the coverage percentage.